

Examen Demo

```
# Your imports HERE !!!!
import numpy as np
import pandas as pd
import math
```

01

- Dado un diccionario con una lista de nombres de películas y el año concatenado, crear una función que devuelva un DataFrame con dos columnas:
 - Columna 1, el nombre de la película sin el año
 - Columna 2, el año de la película

```
movies = {'Name': ["Matrix - 1999", "Cadena Perpetua - 1994",
"Parasitos - 2019", "Vengadores, Endgame - 2019", "Interstellar -
2014",
                "Django Desencadenado - 2012", "El Padrino - 1972",
"El Caballero Oscuro - 2008", "Erase Una Vez En America - 1984",
                "El Senyor de los Anillos, La Comunidad del Anillo
- 2001"]}
expected_output = pd.DataFrame({'Name':["Matrix", "Cadena Perpetua",
"Parasitos", "Vengadores, Endgame", "Interstellar", "Django
Desencadenado",
                "El Padrino", "El Caballero
Oscuro", "Erase Una Vez En America", "El Senyor de los Anillos, La
Comunidad del Anillo"],
                                'Year':["1999", "1994", "2019",
"2019", "2014", "2012", "1972", "2008", "1984", "2001"]})
output = extract_year(movies) # This will call your function
extract_year
display(output)
assert(expected_output.equals(output)) # This will fail if the result
is not as expected
```

	Name	Year
0	Matrix	1999
1	Cadena Perpetua	1994
2	Parasitos	2019
3	Vengadores, Endgame	2019
4	Interstellar	2014
5	Django Desencadenado	2012
6	El Padrino	1972
7	El Caballero Oscuro	2008
8	Erase Una Vez En America	1984
9	El Senyor de los Anillos, La Comunidad del Anillo	2001

```

# Your solution HERE !!!!
def extract_year(movies):
    df_movies = pd.DataFrame(movies)

    def split_year(title):
        index = title.index('-')
        return title[index:].replace('-', '').strip()

    df_movies['Year'] = df_movies['Name'].apply(split_year)

    def remove_year(title):
        index = title.index('-')
        return title[:index-1].strip()

    df_movies['Name'] = df_movies['Name'].apply(remove_year)

    return df_movies

def extract_year(movies):
    movies = pd.DataFrame.from_dict(movies)
    movies['Year'] = movies['Name'].str.split('-').str[1].str.strip()
    movies['Name'] = movies['Name'].str.split('-').str[0].str.strip()
    return movies

def extract_year(movies):
    df_movies=pd.DataFrame(movies)
    df_movies['Year'] = df_movies['Name'].apply(lambda x : x.split('-')
[1].strip())
    df_movies['Name'] = df_movies['Name'].apply(lambda x : x.split('-')
[0].strip())
    return df_movies

```

02

- Crear una función que reciba un array de NumPy y devuelva otro en el cual estén marcados como True los elementos duplicados (a partir de la segunda ocurrencia), y como False los no repetidos o las primeras ocurrencias de los duplicados.

```

numbers = np.array([4, 4, 4, 3, 8, 1, 9, 6, 1, 5, 9, 4, 0, 2])
expected_output = np.array([False, True, True, False, False, False,
False, False, True, False, True, True, False, False])
output = find_duplicates(numbers)    # This calls your function
find_duplicates
print(output)
assert(np.array_equal(expected_output, output))    # This will fail if
the result is not as expected

0      False
1       True
2       True
3      False

```

```

4      False
5      False
6      False
7      False
8      True
9      False
10     True
11     True
12     False
13     False
dtype: bool

# Your solution HERE !!!!
def find_duplicates(numbers):
    serie = pd.Series(numbers)
    return serie.duplicated()

def find_duplicates(array):
    unicos = np.unique(array, return_index=True)[1]#devuelve dos
arrays , uno con los valores unicos y otro con los indices.elegimos
solo el segundo para guardar en la variable
    boolean = np.full(array.shape, True)
    boolean[unicos]=False
    return boolean

def find_duplicates(numbers):
    _, i = np.unique(numbers, return_index=True)
    res = np.zeros_like(numbers, dtype=bool)
    res[i] = True
    res = np.invert(res)
    return res

```

03

- Resolución de dos sistemas de ecuaciones
 - \$ A : \begin{cases} -9x + 4y = 20 \\ -7y + 16x = 80 \end{cases} \$
 - \$ B : \begin{cases} x - 2y + 3z = 7 \\ 2x + y + z = 4 \\ -3x + 2y - 2z = -10 \end{cases} \$
- Verifica por código los resultados

```

# Your solution HERE !!!!
a = np.array([ [-9,4], [16,-7] ])
b = np.array([20,80])

x,y = np.linalg.solve(a,b)
print(f'x={x} y={y}')

print(math.isclose(-9*x + 4*y, 20))
print(math.isclose(-7*y + 16*x, 80))

```

```

x=460.0 y=1040.0
True
True

# Your solution HERE !!!!
a = np.array([ [1,-2,3], [2,1,1], [-3,2,-2]])
b = np.array([7,4,-10])

x,y,z = np.linalg.solve(a,b)
print(f'x={x} y={y} z={z}')

print(math.isclose(x - 2*y + 3*z, 7))
print(math.isclose(2*x + y + z, 4))
print(math.isclose(-3*x + 2*y - 2*z, -10))

```

04

- Crear un objeto Series con 10 elementos, que contenga más de un tipo básico, con índices de tipo string
- Demuestra que los datos con índices posicionales son los mismos que los obtenidos con índices semánticos, seleccionando a través de slicing, al menos 4 elementos de la serie anterior

```

# Your solution HERE !!!!
serie = pd.Series([1,5,3,4.5,True,"acelga"], index=list('abcdef'))
serie.loc['d'] == serie.iloc[4]

a    True
b    True
c    True
d    True
dtype: bool

```

05

- Crear un DataFrame de dimensiones 10x10, con números aleatorios en el rango [0,20]
- Reemplazar todos aquellos elementos menores de 5 por NaN
- Averiguar cuantos elementos tienen Nan por fila

```

# Your solution HERE !!!!
df = pd.DataFrame(np.random.randint(21, size=(10,10)))
display(df)

df.where(df>=5,np.nan, inplace=True)
display(df)

print(df.isnull().sum(axis=1))

arr = np.random.randint(0, 21, size=(10,10))
dataset = pd.DataFrame(arr)

```

```
dataset[dataset<5] = np.nan
dataset.isnull().sum(axis=1)
```

```
0    4
1    6
2    3
3    1
4    2
5    3
6    1
7    2
8    3
9    2
dtype: int64
```

06

- Dada una lista de elementos, crea una función que devuelva un dataframe sin los elementos duplicados

```
serie = ['a','b','c','a','c','a','g']
expected_output = pd.DataFrame(['a','b','c','g'])
output = remove_duplicates(serie)  # This will call your function
remove_duplicates
assert(expected_output.equals(output))  # This will fail if the result
is not as expected

# Your solution HERE !!!!
def remove_duplicates(serie):
    return pd.DataFrame(np.unique(serie))

def remove_duplicates(serie):
    serie= pd.DataFrame(sorted(set(serie)))
    return serie

def remove_duplicates(serie_dup):
    return
pd.DataFrame(serie_dup).drop_duplicates().reset_index(drop=True)

def remove_duplicates(lista):
    output = []
    for x in lista:
        if x not in output:
            output.append(x)
    return pd.DataFrame(output)

def remove_duplicates(lista):
    return
pd.DataFrame(set(lista)).sort_values(by=0).reset_index(drop=True)
```

07 - 1pt

- Escribir un generador `frange` que simule el comportamiento de la función `arange` de NumPy
- Esta función cada vez que se invoque debe devolver valores espaciados uniformemente a partir de un valor inicial. Se debe proveer a dicha función un valor inicial para la secuencia de números y un valor para el step
- Debe soportar valores numéricos en coma flotante
- La función potencialmente se puede llamar un número infinito de veces
- Usando plain Python, sin ninguna librería

```
import math

init = 10.6
step = 0.8
g = frange(init, step) # This calls your function frange

iters = 10
numbers = [next(g) for n in range(iters)]
for i, n in enumerate(numbers):
    assert(math.isclose(n, init + (step * i))) # This will fail if the range is not properly created
print(numbers)

[10.6, 11.4, 12.200000000000001, 13.000000000000002,
13.800000000000002, 14.600000000000003, 15.400000000000004,
16.200000000000003, 17.000000000000004, 17.800000000000004]

# Your solution HERE !!!!
def frange(init, step):
    while True:
        yield init
        init += step
```

08

- Crear una función que reciba una Serie de Pandas y devuelva esa misma serie, reemplazando los espacios en blanco por el carácter menos frecuente en dicha serie.

```
serie = pd.Series(list('bba cabc faabba aacbbfe'))
expected_output = pd.Series(list('bbaecabcefaabbaeaacbbfe'))
output = clean_whitespace_replace(serie) # This calls your function clean_whitespace_replace
assert(expected_output.equals(output)) # This will fail if result is not as expected

# Your solution HERE !!!!
def clean_whitespace_replace(serie):
    less_freq = serie.value_counts().idxmin()
    serie[serie==' '] = less_freq
    return serie
```

```
def clean_whitespace_replace(serie):
    element_freq = serie.replace(' ', '').value_counts()
    least_frequent_char = element_freq.index[-1]
    return pd.Series(list("".join(serie.replace(' ',
least_frequent_char))))

def clean_whitespace_replace(serie_rep):
    return serie_rep.replace(' ', serie.value_counts().idxmin())
```